

LAPORAN PRAKTIKUM 5

Struktur Data



RAFKI AHMAD PAGAMANDA

2311533016

Kelas D

Implementasi Single Linked List

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

Implementasi Single Linked List

1. NodeSLL

```
1 package pekan5_2311533016;
2
3 public class NodeSLL_2311533016 {
4     // node bagian data
5     int data_3016;
6
7     // Pointer ke node berikutnya
8     NodeSLL_2311533016 next_3016;
9
10    // Konstruktor menginisialisasi node dengan data
11    public NodeSLL_2311533016(int data_3016) {
12        this.data_3016 = data_3016;
13        this.next_3016 = null;
14    }
15 }
```

Pada class Pasien_2311533016, sy membuat sebuah ADT (Abstract Data Type) yang digunakan untuk menyimpan data pasien pada sistem antrian rumah sakit. Class ini berfungsi sebagai node pada Single Linked List, dimana setiap node menyimpan informasi pasien serta pointer yang menunjuk ke node berikutnya.

Di dalam class terdapat beberapa atribut yaitu namaPasien_3016, keluhan_3016, dan nomorAntrian_3016. Variabel tersebut digunakan untuk menyimpan nama pasien, jenis keluhan atau penyakit pasien, serta nomor antrian pasien. Selain itu terdapat pointer next_3016 yang berfungsi untuk menghubungkan node saat ini dengan node pasien berikutnya pada linked list.

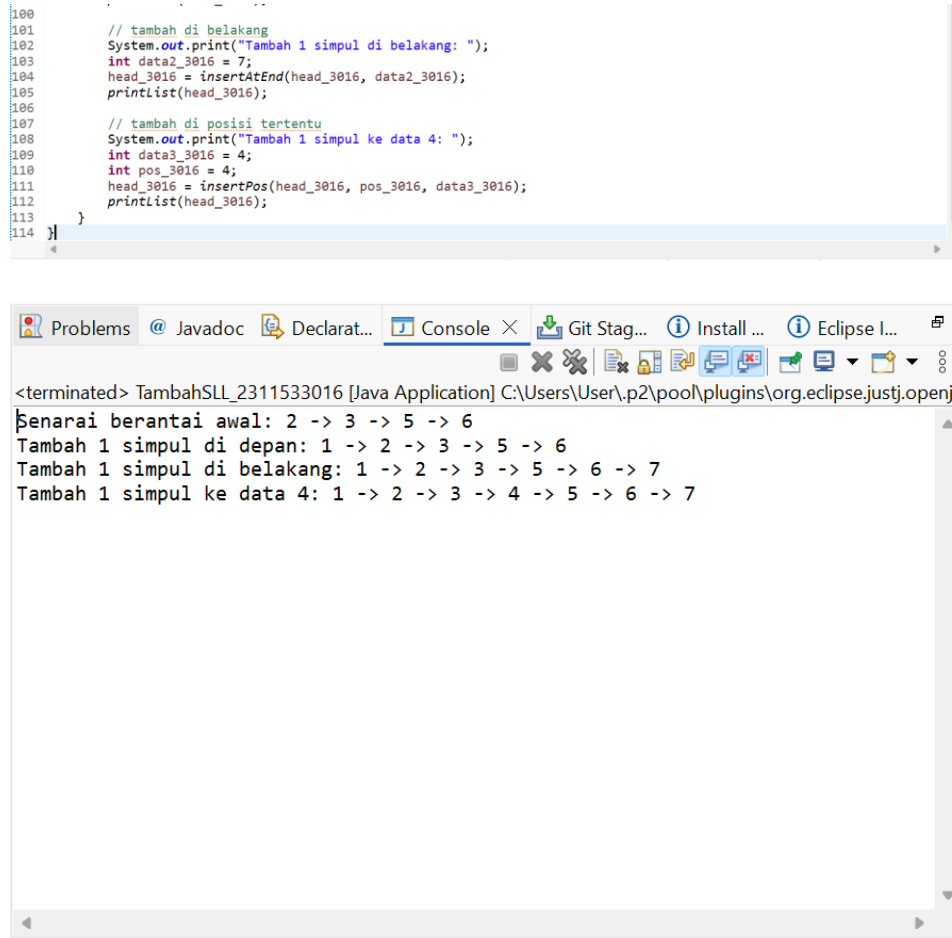
Pada constructor, sy menginisialisasi seluruh atribut ketika object pasien dibuat. Data nama pasien, keluhan, dan nomor antrian langsung dimasukkan melalui parameter constructor sehingga object dapat langsung digunakan. Pointer next_3016 diisi null sebagai tanda bahwa node belum terhubung ke node lain.

Class ini juga memiliki method getter dan setter. Getter digunakan untuk mengambil data dari atribut, sedangkan setter digunakan untuk mengubah nilai atribut maupun pointer next_3016. Dengan adanya getter dan setter, pengolahan data pasien menjadi lebih mudah dan terstruktur.

Output dari class ini sendiri tidak langsung terlihat karena class hanya digunakan sebagai pembentuk object pasien dan node linked list. Namun class ini menjadi dasar utama agar data pasien dapat diproses pada program antrian rumah sakit.

2. TambahSLL

```
1 package pekan5_2311533016;
2
3 public class TambahSLL_2311533016 {
4
5     public static NodeSLL_2311533016 insertAtFront(NodeSLL_2311533016 head_3016, int value_3016) {
6         NodeSLL_2311533016 new_node_3016 = new NodeSLL_2311533016(value_3016);
7         new_node_3016.next_3016 = head_3016;
8         return new_node_3016;
9     }
10
11     public static NodeSLL_2311533016 insertAtEnd(NodeSLL_2311533016 head_3016, int value_3016) {
12         NodeSLL_2311533016 newNode_3016 = new NodeSLL_2311533016(value_3016);
13
14         if (head_3016 == null) {
15             return newNode_3016;
16         }
17
18         NodeSLL_2311533016 last_3016 = head_3016;
19
20         while (last_3016.next_3016 != null) {
21             last_3016 = last_3016.next_3016;
22         }
23
24         last_3016.next_3016 = newNode_3016;
25
26         return head_3016;
27     }
28
29     static NodeSLL_2311533016 GetNode(int data_3016) {
30         return new NodeSLL_2311533016(data_3016);
31     }
32
33     static NodeSLL_2311533016 insertPos(NodeSLL_2311533016 headNode_3016, int position_3016, int value_3016) {
34         NodeSLL_2311533016 head_3016 = headNode_3016;
35
36         if (position_3016 < 1)
37             System.out.print("Invalid position");
38
39         if (position_3016 == 1) {
40             NodeSLL_2311533016 new_node_3016 = new NodeSLL_2311533016(value_3016);
41             new_node_3016.next_3016 = head_3016;
42             return new_node_3016;
43         } else {
44             while (position_3016-- != 0) {
45                 if (position_3016 == 1) {
46                     NodeSLL_2311533016 newNode_3016 = GetNode(value_3016);
47                     NodeSLL_2311533016 newHead_3016 = GetNode(value_3016);
48
49                     newNode_3016.next_3016 = headNode_3016.next_3016;
50                     headNode_3016.next_3016 = newNode_3016;
51                     break;
52                 }
53                 headNode_3016 = headNode_3016.next_3016;
54             }
55
56             if (position_3016 != 1)
57                 System.out.print("Posisi di luar jangkauan");
58
59             return head_3016;
60         }
61     }
62
63     public static void printList(NodeSLL_2311533016 head_3016) {
64         NodeSLL_2311533016 curr_3016 = head_3016;
65
66         while (curr_3016.next_3016 != null) {
67             System.out.print(curr_3016.data_3016 + " -> ");
68             curr_3016 = curr_3016.next_3016;
69         }
70
71         if (curr_3016.next_3016 == null) {
72             System.out.print(curr_3016.data_3016);
73         }
74
75         System.out.println();
76     }
77
78     public static void main(String[] args) {
79         // buat linked list 2->3->5->6
80         NodeSLL_2311533016 head_3016 = new NodeSLL_2311533016(2);
81         head_3016.next_3016 = new NodeSLL_2311533016(3);
82         head_3016.next_3016.next_3016 = new NodeSLL_2311533016(5);
83         head_3016.next_3016.next_3016.next_3016 = new NodeSLL_2311533016(6);
84
85         // cetak list asli
86         System.out.print("Senarai berantai awal: ");
87         printList(head_3016);
88
89         // tambah di depan
90         System.out.print("Tambah 1 simpul di depan: ");
91         int data_3016 = 1;
92         head_3016 = insertAtFront(head_3016, data_3016);
93         printList(head_3016);
94     }
95 }
```



```
100
101 // tambah di belakang
102 System.out.print("Tambah 1 simpul di belakang: ");
103 int data2_3016 = 7;
104 head_3016 = insertAtEnd(head_3016, data2_3016);
105 printList(head_3016);
106
107 // tambah di posisi tertentu
108 System.out.print("Tambah 1 simpul ke data 4: ");
109 int data3_3016 = 4;
110 int pos_3016 = 4;
111 head_3016 = insertPos(head_3016, pos_3016, data3_3016);
112 printList(head_3016);
113 }
114 }
```

<terminated> TambahSLL_2311533016 [Java Application] C:\Users\User\p2\pool\plugins\org.eclipse.justj.openjc
Benarai berantai awal: 2 -> 3 -> 5 -> 6
Tambah 1 simpul di depan: 1 -> 2 -> 3 -> 5 -> 6
Tambah 1 simpul di belakang: 1 -> 2 -> 3 -> 5 -> 6 -> 7
Tambah 1 simpul ke data 4: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7

Pada class TambahSLL_2311533016, program digunakan untuk melakukan penambahan data pada Single Linked List. Program menyediakan beberapa fungsi, yaitu menambahkan node di depan list, di belakang list, dan pada posisi tertentu.

Method insertAtFront() digunakan untuk menambahkan node baru di bagian depan linked list. Node baru akan menjadi head baru, sedangkan node lama dipindahkan menjadi node berikutnya.

Method insertAtEnd() digunakan untuk menambahkan node di bagian akhir linked list. Program akan menelusuri node satu per satu hingga mencapai node terakhir, kemudian node baru dihubungkan pada bagian akhir list.

Method insertPos() digunakan untuk menambahkan node pada posisi tertentu. Program akan mencari posisi yang diminta, lalu menyisipkan node baru di antara node yang sudah ada sebelumnya.

Selain itu, terdapat method printList() yang digunakan untuk menampilkan seluruh isi linked list ke layar. Output ditampilkan dengan tanda panah (-->) agar hubungan antar node terlihat jelas.

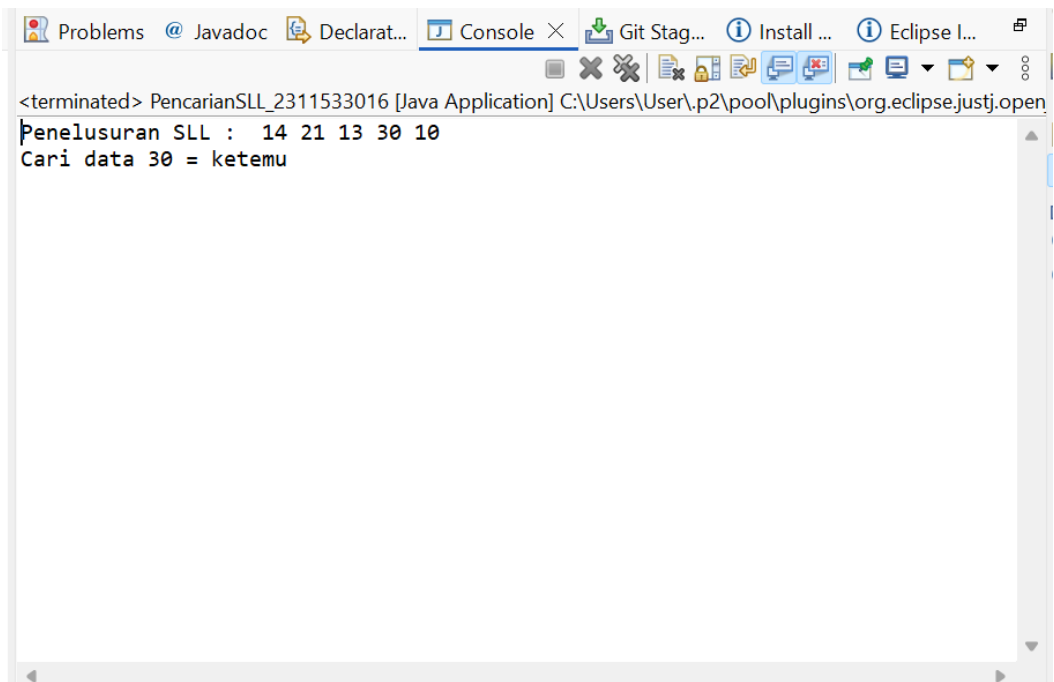
Pada method main, program membuat linked list awal dengan data 2 -> 3 -> 5 -> 6, kemudian dilakukan beberapa proses penambahan data, yaitu:

- menambah data 1 di depan,
- menambah data 7 di belakang,
- dan menambah data 4 pada posisi tertentu.

Hasil output menunjukkan bahwa data berhasil ditambahkan sesuai posisi yang diinginkan sehingga struktur linked list berubah menjadi lebih lengkap.

3. PencarianSLL

```
1 package pencarian_sll;
2
3 public class PencarianSLL_2311533016 {
4
5     static boolean searchKey(NodeSLL_2311533016 head_3016, int key_3016) {
6
7         NodeSLL_2311533016 curr_3016 = head_3016;
8
9         while (curr_3016 != null) {
10             if (curr_3016.data_3016 == key_3016)
11                 return true;
12
13             curr_3016 = curr_3016.next_3016;
14         }
15
16         return false;
17     }
18
19     public static void traversal(NodeSLL_2311533016 head_3016) {
20
21         NodeSLL_2311533016 curr_3016 = head_3016;
22
23         while (curr_3016 != null) {
24             System.out.print(" " + curr_3016.data_3016);
25             curr_3016 = curr_3016.next_3016;
26         }
27
28         System.out.println();
29     }
30
31     public static void main(String[] args) {
32
33         NodeSLL_2311533016 head_3016 = new NodeSLL_2311533016(14);
34         head_3016.next_3016 = new NodeSLL_2311533016(21);
35         head_3016.next_3016.next_3016 = new NodeSLL_2311533016(13);
36         head_3016.next_3016.next_3016.next_3016 = new NodeSLL_2311533016(30);
37         head_3016.next_3016.next_3016.next_3016.next_3016 = new NodeSLL_2311533016(10);
38
39         System.out.print("Penelusuran SLL : ");
40         traversal(head_3016);
41
42         // data yang akan dicari
43         int key_3016 = 30;
44
45         System.out.print("Cari data " + key_3016 + " = ");
46
47         if (searchKey(head_3016, key_3016))
48             System.out.println("ketemu");
49         else
50             System.out.println("tidak ada");
51     }
52 }
```



```
<terminated> PencarianSLL_2311533016 [Java Application] C:\Users\User\p2\pool\plugins\org.eclipse.justj.open
Penelusuran SLL : 14 21 13 30 10
Cari data 30 = ketemu
```

Pada class PencarianSLL_2311533016, program digunakan untuk melakukan proses pencarian data pada Single Linked List. Program akan memeriksa setiap node secara berurutan hingga menemukan data yang dicari.

Method searchKey() digunakan untuk mencari nilai tertentu pada linked list. Program melakukan traversal mulai dari head hingga node terakhir. Jika data ditemukan, method akan mengembalikan nilai true, sedangkan jika data tidak ditemukan maka akan mengembalikan false.

Method traversal() digunakan untuk menampilkan seluruh isi linked list dari awal hingga akhir. Data ditampilkan secara berurutan sehingga pengguna dapat melihat isi list sebelum proses pencarian dilakukan.

Pada method main, program membuat linked list dengan data:

14 -> 21 -> 13 -> 30 -> 10

Kemudian program mencari data 30 pada linked list tersebut. Karena data ditemukan, output yang muncul adalah:

Cari data 30 = ketemu

Output tersebut menunjukkan bahwa proses traversal dan pencarian berhasil dilakukan dengan baik menggunakan metode Single Linked List.

4. HapusSLL

```
1 package pekan5_2311533016;
2
3 public class HapusSLL_2311533016 {
4
5     // hapus head
6     public static NodeSLL_2311533016 deleteHead(NodeSLL_2311533016 head_3016) {
7         if (head_3016 == null)
8             return null;
9
10        head_3016 = head_3016.next_3016;
11        return head_3016;
12    }
13
14    // hapus node terakhir
15    public static NodeSLL_2311533016 removeLastNode(NodeSLL_2311533016 head_3016) {
16
17        if (head_3016 == null)
18            return null;
19
20        if (head_3016.next_3016 == null)
21            return null;
22
23        NodeSLL_2311533016 secondLast_3016 = head_3016;
24
25        while (secondLast_3016.next_3016.next_3016 != null) {
26            secondLast_3016 = secondLast_3016.next_3016;
27        }
28
29        secondLast_3016.next_3016 = null;
30
31        return head_3016;
32    }
33
34    // hapus node berdasarkan posisi
35    public static NodeSLL_2311533016 deleteNode(NodeSLL_2311533016 head_3016, int position_3016) {
36
37        NodeSLL_2311533016 temp_3016 = head_3016;
38        NodeSLL_2311533016 prev_3016 = null;
39
40        if (temp_3016 == null)
41            return head_3016;
42
43        if (position_3016 == 1) {
44            head_3016 = temp_3016.next_3016;
45            return head_3016;
46        }
47
48        for (int i_3016 = 1; temp_3016 != null && i_3016 < position_3016; i_3016++) {
49            prev_3016 = temp_3016;
50            temp_3016 = temp_3016.next_3016;
51        }
52    }
53 }
```

```

51     }
52
53     if (temp_3016 != null)
54         prev_3016.next_3016 = temp_3016.next_3016;
55     else
56         System.out.println("Data tidak ada");
57     return head_3016;
58 }
59
60 // print list
61 public static void printList(NodeSLL_2311533016 head_3016) {
62     NodeSLL_2311533016 curr_3016 = head_3016;
63
64     while (curr_3016.next_3016 != null) {
65         System.out.print(curr_3016.data_3016 + " -> ");
66         curr_3016 = curr_3016.next_3016;
67     }
68
69     if (curr_3016.next_3016 == null) {
70         System.out.print(curr_3016.data_3016);
71     }
72     System.out.println();
73 }
74
75 // ===== MAIN =====
76 public static void main(String[] args) {
77
78     // buat linked list: 1 -> 2 -> 3 -> 4 -> 5 -> 6
79     NodeSLL_2311533016 head_3016 = new NodeSLL_2311533016(1);
80     head_3016.next_3016 = new NodeSLL_2311533016(2);
81     head_3016.next_3016.next_3016 = new NodeSLL_2311533016(3);
82     head_3016.next_3016.next_3016.next_3016 = new NodeSLL_2311533016(4);
83     head_3016.next_3016.next_3016.next_3016.next_3016 = new NodeSLL_2311533016(5);
84     head_3016.next_3016.next_3016.next_3016.next_3016.next_3016 = new NodeSLL_2311533016(6);
85
86     // tampilkan awal
87     System.out.println("List awal:");
88     printList(head_3016);
89
90     // hapus head
91     head_3016 = deleteHead(head_3016);
92     System.out.println("List setelah head dihapus:");
93     printList(head_3016);
94
95     // hapus node terakhir
96     head_3016 = removeLastNode(head_3016);
97     System.out.println("List setelah simpul terakhir dihapus:");
98     printList(head_3016);
99
100    // hapus posisi ke-2
101    int position_3016 = 2;
102    head_3016 = deleteNode(head_3016, position_3016);
103
104    System.out.println("List setelah posisi 2 dihapus:");
105    printList(head_3016);
106 }
107 }

```

Problems @ Javadoc Declarat... Console X Git Stag... Install ... Eclipse I...

<terminated> HapusSLL_2311533016 [Java Application] C:\Users\User\p2\pool\plugins\org.eclipse.justj.openjdk

```

List awal:
1 -> 2 -> 3 -> 4 -> 5 -> 6
List setelah head dihapus:
2 -> 3 -> 4 -> 5 -> 6
List setelah simpul terakhir dihapus:
2 -> 3 -> 4 -> 5
List setelah posisi 2 dihapus:
2 -> 4 -> 5

```

Pada class HapusSLL_2311533016, program digunakan untuk melakukan penghapusan data pada Single Linked List. Program menyediakan beberapa fungsi penghapusan, yaitu menghapus node pertama, node terakhir, dan node pada posisi tertentu.

Method deleteHead() digunakan untuk menghapus node pertama pada linked list. Program memindahkan posisi head ke node berikutnya sehingga node awal tidak lagi terhubung dengan list.

Method removeLastNode() digunakan untuk menghapus node terakhir. Program menelusuri linked list hingga mencapai node sebelum node terakhir, kemudian pointer next diubah menjadi null sehingga node terakhir terhapus. Method deleteNode() digunakan untuk menghapus node pada posisi tertentu. Program akan mencari posisi node yang ingin dihapus, lalu menghubungkan node sebelumnya dengan node setelahnya agar node target keluar dari linked list. Method printList() digunakan untuk menampilkan isi linked list setelah proses penghapusan dilakukan.

Pada method main, program membuat linked list awal:

1 -> 2 -> 3 -> 4 -> 5 -> 6

Kemudian program melakukan beberapa proses penghapusan, yaitu:

- menghapus head,
- menghapus node terakhir,
- dan menghapus node pada posisi ke-2.

Output program menunjukkan perubahan isi linked list setelah setiap proses penghapusan dilakukan. Hal ini membuktikan bahwa operasi delete pada Single Linked List berhasil berjalan sesuai fungsi yang dibuat.

Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan struktur data yang digunakan untuk menyimpan data secara berantai menggunakan node yang saling terhubung melalui pointer. Pada praktikum ini telah dilakukan beberapa operasi dasar Single Linked List yaitu penambahan data, pencarian data, traversal, dan penghapusan data.